

RAPPORT DE STAGE

Camille Kluck

BUT MMI 2^{ème} Année



PROMOD

Ludovic Lefevre

Gilles Audemard

IUT
Institut
Universitaire
Technologie
LENS

Année universitaire 2024-2025

Remerciements

À l'issue de mon stage, je tiens à exprimer ma gratitude à toutes les personnes qui m'ont aidée et soutenue, et qui ont fait de cette expérience un moment particulièrement enrichissant.

Je remercie tout d'abord mon maître de stage, M. Ludovic Lefevre, responsable de domaine chez PROMOD, pour la qualité de son suivi et l'encadrement dont il a fait preuve tout au long de mon stage.

Je souhaite également remercier l'équipe de réalisation du site e-commerce pour m'avoir accueillie chaleureusement et accompagnée tout au long de cette période.

Je remercie plus particulièrement Nicolas Delbouve, Julien Dutrieux et Henri Boulnois pour leur accompagnement constant, leurs conseils, leur expertise et leurs retours constructifs sur le travail que j'ai réalisé.

Je tiens aussi à remercier Julien Dubus et Aurélien Bourdon, collaborateurs du projet, pour la mise en place de l'API Vision, composante essentielle du projet, ainsi que pour leurs retours précieux sur mes contributions.

Enfin, du côté de l'IUT de Lens, je remercie M. Audemard pour le suivi attentif de mon stage durant ces deux mois, ainsi que Mme Charnay dans son rôle pour la coordination des stages.

Merci à tous pour ce très beau et enrichissant stage.

PS : Merci à toute l'équipe e-commerce pour le Laser Game.

SOMMAIRE

I.	INTRODUCTION	5
II.	DÉVELOPPEMENT.....	6
1	Présentation et historique de l'entreprise.....	6
A.	Historique.....	6
B.	Spécificités de PROMOD	6
C.	La cible	7
D.	Les différents services de la DSI	7
E.	Le service	8
2	Mes missions.....	8
A.	La gestion de l'équipe et du projet	8
B.	Environnement technique de l'entreprise	9
C.	Les bases	10
D.	Le projet.....	10
1.	<i>Le Fashion Seeker avec les produits présents sur le site.....</i>	<i>11</i>
2.	<i>Le Fashion Seeker avec l'upload d'un fichier</i>	<i>16</i>
3.	<i>Les tests.....</i>	<i>17</i>
III.	CONCLUSION	19

I. INTRODUCTION

Dans le cadre de ma formation du BUT MMI (Métiers du Multimédia et de l'Internet), j'ai effectué un stage de 2 mois dans l'entreprise PROMOD du 22 avril 2025 au 26 juin 2025. J'ai été accueillie dans l'entreprise par Ludovic Lefevre qui est responsable de domaine, qui m'a intégrée dans l'équipe de développement du site e-commerce de l'entreprise.

PROMOD est une marque française spécialisée dans le prêt-à-porter féminin. Elle est engagée dans une mode plus inclusive et responsable et valorise la diversité à travers ses actions et ses collections. De plus, l'entreprise est engagée dans le développement de la technologie participant notamment au DevLille qui est un grand événement pour la communauté de la tech avec l'occasion d'échanger autour de thèmes comme le Web ou l'Intelligence artificielle.

Inclue dans l'équipe de développement du site e-commerce, j'ai eu l'occasion de participer à son développement en travaillant notamment sur le projet de l'intégration du « Fashion Seeker ». J'ai dû – entre autres – apprendre de nouvelles compétences en matière de code pour m'adapter au nouvel environnement de travail et renforcer mes compétences déjà acquises dans le domaine, tous en apportant à l'équipe ma réflexion pour mettre en œuvre le projet.

Ainsi, ce stage a aussi été la possibilité pour moi d'en apprendre davantage sur le monde de l'entreprise, en participant plus particulièrement aux rituels agiles SCRUM mis en place dans l'équipe, mais aussi en m'intégrant à la vie d'entreprise en participant à des échanges et à des moments de convivialité organisés par l'équipe.

Afin de mieux assimiler l'environnement dans lequel s'est déroulé mon stage, je commencerai par présenter plus en détail l'entreprise PROMOD, avant de revenir sur les missions qui m'ont été confiées et les compétences que j'ai pu développer au cours de cette expérience.

II. DÉVELOPPEMENT

1 Présentation et historique de l'entreprise

A. Historique

PROMOD est une marque de prêt-à-porter féminine créée en 1975 par Francis-Charles Pollet. C'est une entreprise nordiste qui ouvre son premier magasin à Roubaix.

L'objectif principal de cette entreprise est de proposer aux femmes une mode accessible, de qualité et moderne. Ce qui est en adéquation avec l'année de création de l'entreprise, car 1975 est l'année internationale de la femme.

Aux bouts de quelques années, PROMOD connaît une croissance rapide et s'exporte à l'internationale comme en Espagne ou encore en Allemagne. Mais en 2016, à la suite de la baisse de vente à l'étranger, l'entreprise est obligée de supprimer des emplois.

En 2018, le fils de Francis-Charles Pollet, Julien Pollet, reprend les rênes de l'entreprise et apporte plusieurs changements au cours des prochaines années avec un plan de transformation qui vise à monter en gamme, réduire le nombre de nouveaux produits créés et d'être encore plus à l'écoute des clientes pour donner suite à la forte émergence de la fast-fashion.

Plus récemment, l'entreprise se lance dans la mise en place de pratiques de vente plus durables et en adéquation avec leurs engagements, comme la seconde main avec par exemple la mise en place de corners éphémères ou la vente de chutes de tissu. Et en 2023, ils lancent leur nouveau site d'e-commerce.

B. Spécificités de PROMOD

Pour ce qui est des spécificités de l'entreprise, elle se base d'abord sur une logique de produire moins mais mieux car cette technique marchait dans l'année 2000, mais plus maintenant dû à l'émergence des grands de la fast-fashion.

Ensuite, PROMOD est une entreprise très engagée sur deux points importants. Tout d'abord, le développement durable en créant par exemple le label « PROMOD for Good » avec l'objectif de créer une collection 100 % éco-responsable, et en se dirigeant vers des matières plus responsables et en affichant clairement la composition et la provenance des produits. Malgré tout, en 2025, l'objectif fixé n'étant plus réaliste, ils ont donc arrêté le label fin 2024, mais ils s'engagent sur

l'éco-conception et l'affichage de l'impact environnemental de leurs produits. De plus, l'entreprise essaie de revendre les vêtements dits de seconde main pour éviter le gâchis avec les stocks.

En deuxième point, la marque se pose comme une marque qui habille toutes les femmes, de toute génération, personnalité, morphologie. La marque valorise aussi la diversité en proposant des collections qui s'adaptent à tout le monde. Elle a même brièvement exploré une gamme masculine, témoignant de sa volonté d'élargir son offre, bien que ce projet n'ait pas été poursuivi.

C. La cible

Pour ce qui est de la cible, la marque s'adresse principalement aux femmes en général, avec quand même un style principalement bohème dans les produits qu'elle propose. Mais il y a quand même des produits qui ne s'inscrivent pas principalement dans ce genre-là et qui peuvent convenir à une cible qui n'est pas forcément adepte du style bohème.

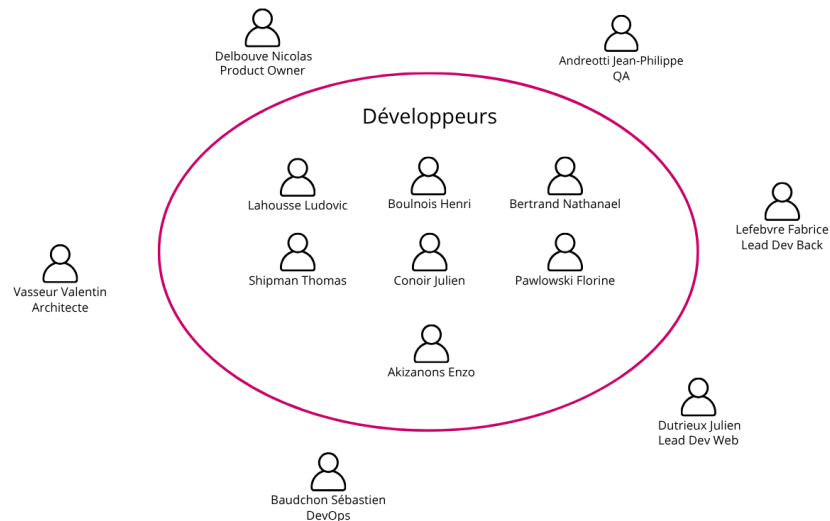
De plus, la marque s'adresse aussi, dans leurs engagements, aux femmes atteintes du cancer du sein ou qui ont subi une mastectomie. En effet, PROMOD a créé la Boutique Rose qui propose une sélection de vêtements correspondant aux problématiques engendrées par le cancer du sein.

D. Les différents services de la DSI

Il y a différents services dans la DSI de l'entreprise, tout d'abord le service client qui est lui-même composé de différents services comme le service magasins, CRM et e-commerce qui est celui dans lequel je suis intégrée mais il y a aussi d'autres services comme le Supply qui regroupe la logistique, les achats, les collections et offres et la gestion commerciale. Puis on peut retrouver le service ventes et catalogues qui contient les ventes et le back office vente. Le service suivant est celui des finances et des ressources humaines avec aussi la gestion administrative des magasins. Puis pour finir, le service de la data. Tous ces services communiquent entre eux et sont indispensables à l'entreprise.

E. Le service

Pour ce qui est du service dans lequel j'ai été accueillie lors de ce stage, il s'agit de l'équipe de réalisation du site web et de l'application e-commerce de PROMOD. Elle est composée de 11 personnes.



Explication sur certains des rôles :

Product Owner : a pour mission de mener à bien un projet en suivant la méthodologie agile ; il retranscrit les besoins métiers en cas d'utilisateurs (US : User Stories) pris en charge par les développeurs ensuite.

Lead Développeur : c'est le manager d'une équipe de développeurs, il coordonne et accompagne les travaux de son équipe de développeurs. Il définit les orientations techniques et est garant des bonnes pratiques.

QA (Quality Assurance) : a pour rôle de tester l'ensemble des fonctionnalités des livrables et donc met en place une stratégie de test et des scénarios permettant de garantir la qualité des livrables.

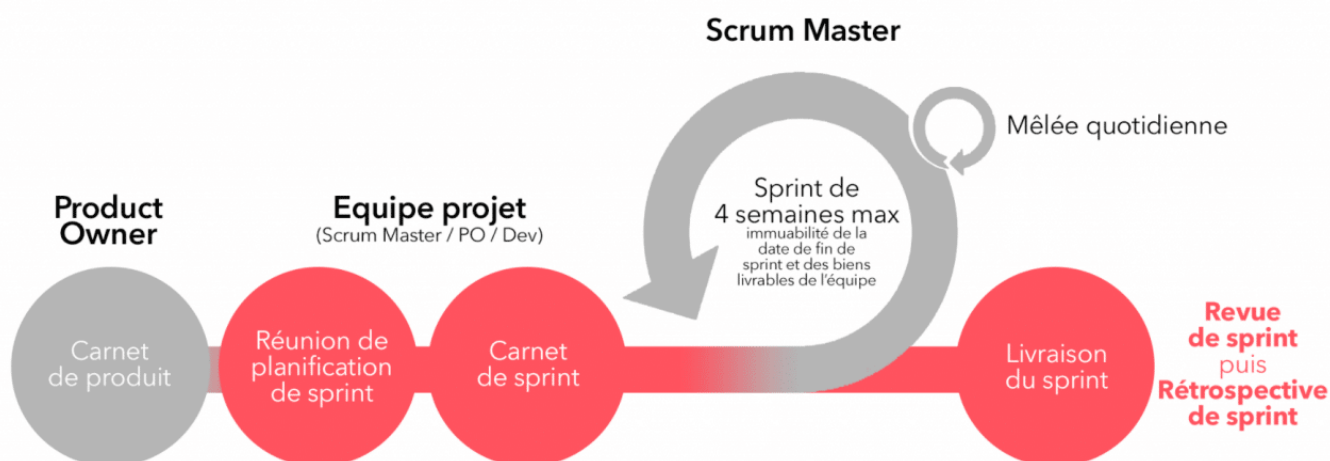
DevOps : a de l'expérience dans l'administration des systèmes, comme la gestion de serveurs, le déploiement de base de données, la surveillance de la sécurité ou encore dans l'application de correctifs aux systèmes.

2 Mes missions

A. La gestion de l'équipe et du projet

Au début de mon stage, on m'a directement inclus dans l'équipe e-commerce, ce qui m'a donné l'opportunité de participer aux différents rituels mis en place au sein de l'équipe. Ils permettent d'organiser et de distribuer les tâches et les projets en

fonction des différents membres de l'équipe. Il y a plusieurs réunions faisant partie de la méthodologie agile SCRUM, tout d'abord toutes les 3 semaines est mis en place un sprint planning où toutes les US¹ à faire au cours des 3 prochaines semaines sont exposées, de plus toutes les US sont notées en fonction de leurs difficultés et du temps que cela va prendre pour les réaliser. Ensuite, j'ai participé tous les jours au Daily Meeting, où pendant 15 à 20 minutes tous les membres de l'équipe expliquent ce qu'ils ont fait le jour d'avant et ce qu'ils vont faire aujourd'hui, mais le Daily sert aussi à exposer les problèmes rencontrés lors du développement d'une fonctionnalité et permet de poser un moment pour aider à fixer les problèmes.



B. Environnement technique de l'entreprise

Pour mettre en place les bases de mon stage, j'ai dû m'adapter aux technologies utilisées par l'entreprise. J'ai donc dû me documenter sur plusieurs choses tout en découvrant le code de l'entreprise ce qui m'a bien servi de modèle tout au long du stage. Le framework utilisé par l'entreprise est Nuxt 3 qui est un framework de Vuejs qui est lui aussi un framework qui utilise le JavaScript et/ou le TypeScript. J'ai donc dû me renseigner aussi sur celui-ci.

J'ai aussi eu l'aide de l'équipe e-commerce pour m'expliquer les bases du code et ce qu'il avait mis en place dans le projet, pour avoir des exemples plus concrets qui me permettaient une meilleure compréhension.

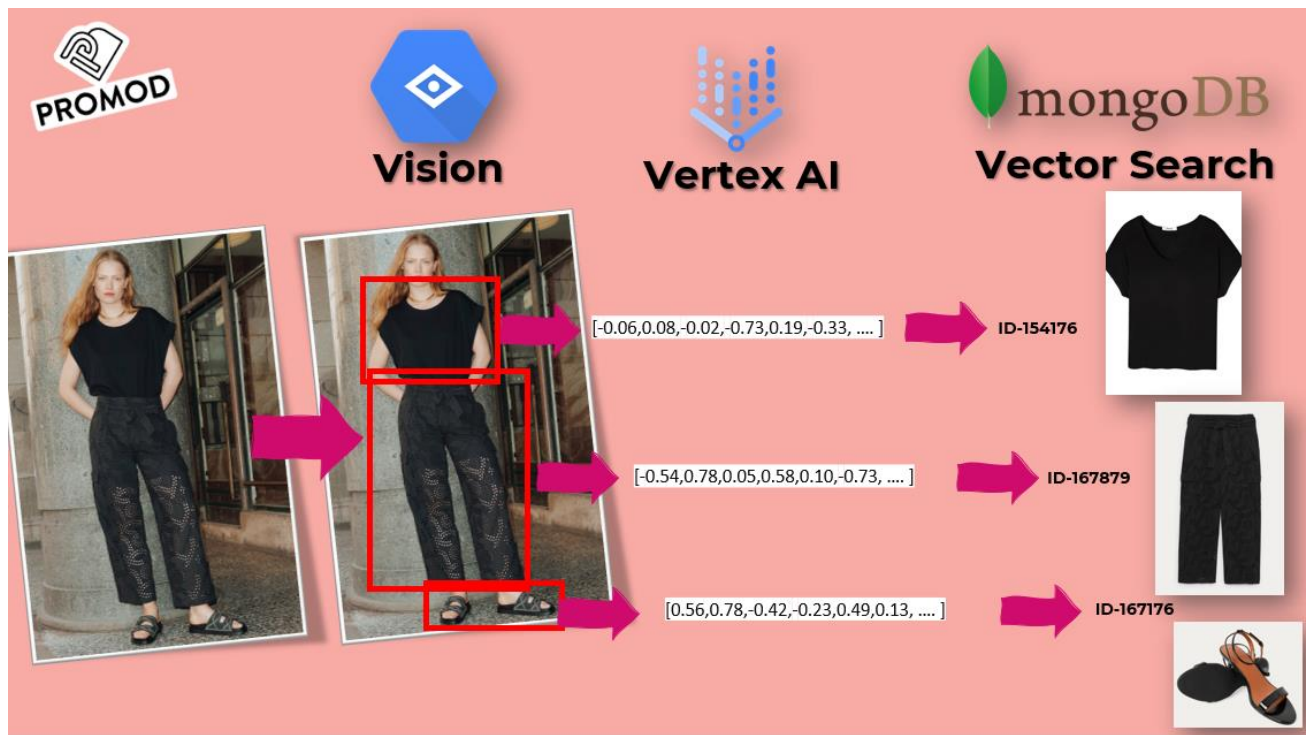
¹US : un récit utilisateur ou « user stories » est une description d'un besoin ou d'une attente d'un utilisateur pour déterminer les nouvelles fonctionnalités à développer.

C. Les bases

Pour ce qui est de ce que j'ai concrètement appris au niveau des bases, c'est tout d'abord la structure de fichiers de Nuxt 3, qui peut, dans certains de ses aspects, ressembler à ce qu'on a avec React comme les composants que l'on retrouve souvent dans les structures de dossiers React, mais, Nuxt est un peu moins libre au niveau de la structure de fichiers que React pour une raison toute particulière, c'est que les routes sont faites automatiquement sous Nuxt3 grâce à la structure de dossiers. Ensuite, j'ai dû me renseigner un peu sur le système de récupération de données, qui est pour Nuxt 3 pas très difficile à comprendre et plus ou moins similaire à Axios (ci-dessous Annexe 1 ci-dessous). Puis dans les fichiers, il y a une structure particulière en 3 parties, la première étant la partie `<script setup lang='ts'>` qui contient tout le TypeScript ou le JavaScript en fonction du langage, la deuxième partie est le `<template>` qui gère l'affichage et la troisième partie est le style qui est très souvent scoped dans les composants, c'est-à-dire qu'il ne va s'appliquer qu'au composant dans lequel il est. Une autre propriété intéressante qui s'utilise au niveau du SCSS, c'est « `:deep` » qui permet de modifier le style d'un composant depuis un autre, cela permet de modifier des composants en fonction de cas particulier, dans le projet, j'en ai utilisé une ou deux fois pour par exemple définir la taille de la modale dans la première version.

D. Le projet

Pour le projet que j'ai eu à faire tout au long du stage, on m'a demandé d'implémenter une fonctionnalité appelée le « Fashion Seeker » qui était déjà présente dans l'application et qui sert à détecter les vêtements dans une image. Quand les vêtements sont détectés grâce à Vision AI cela les envoie à une IA appelée Vertex AI qui va déterminer des vecteurs dans les objets détectés sur l'image et va nous renvoyer les produits qui ressemblent le plus à l'image que l'on a envoyée. J'ai dû lors de ce stage concevoir deux versions de l'interface du « Fashion Seeker », une avec les images des produits disponibles sur le site, et une à côté de la barre de recherche pour importer ses propres images.



1. Le Fashion Seeker avec les produits présents sur le site

J'ai tout d'abord commencé par développer la partie du Fashion Seeker où on peut rechercher les produits similaires grâce à des produits déjà présents sur le site web. Pour cela, on m'a donné une US où l'on me présente ce que je dois faire, la première étape de cette US étant de faire en sorte que le Fashion Seeker dépende d'un paramètre « fashionSeeker », la deuxième étape consistait en l'affichage du bouton qui réalisera la recherche visuelle sur le produit. Pour ce qui est de la troisième étape de cette US, le but était de créer un service et une fonction qui récupère l'URL de l'image au format Grand Zoom, ensuite l'URL est envoyée dans l'API Vision qui utilise deux fonctionnalités de Vision AI de Google pour détecter les objets dans l'image, et cela renvoie une liste de produits similaires sous la forme d'identifiant de produit, puis grâce à ces identifiants on fait un appel au catalogue regroupant tous les produits pour récupérer les produits similaires. Enfin, pour la quatrième et dernière étape de cette US, on affiche les produits similaires dans un composant modal qui s'ouvre quand on clique sur le bouton de la recherche visuelle.

Grâce à cette US, j'ai pu avoir une ligne conductrice pour réaliser ce projet donc j'ai tout d'abord commencé par coder le bouton car je ne pouvais pas faire la première étape n'ayant pas accès à la base de données pour rajouter le paramètre. J'ai donc commencé par rajouter le bouton du fashion Seeker ce qui ne m'a pas pris longtemps. Cependant, quand j'ai commencé à coder la modale, j'ai voulu séparer le composant du bouton et celui de la modale car le produit était considéré comme un lien donc j'ai dû sortir la modale du lien pour éviter un problème qui transformait la modale en lien. J'ai donc dû utiliser un événement personnalisé « emit » pour

pouvoir ouvrir la modale qui était située dans un autre composant. Ensuite, quand le paramètre « fashionSeeker » a été rajouté dans la base de données et la liste des paramètres dans le code, j'ai pu le rajouter comme condition pour l'affichage du bouton.

Ensuite pour bien pouvoir coder la modale j'ai dû créer un service (Voir Annexe 5 ci-dessous), qui est un composables qui permet de faire un appel API. Ici on appellera l'API Vision de l'entreprise. Donc j'ai dans ce service créé une fonction asynchrone qui peut prendre en paramètre deux choses « filename » et « urlImage ». Ici on ne s'est servi que de l'urlImage que l'on envoie comme paramètre dans l'url en méthode POST.

À la suite de cela j'ai pu commencer à coder la modale. J'ai donc d'abord passé à ce composant le produit qui est désigné par la recherche visuelle par le moyen d'une props et qui me servira pour la fonction qui effectuera la recherche. J'ai pu alors créer la fonction « fetchSimilarProduct » qui récupère les produits similaires à l'image envoyée. J'ai d'abord appelé la fonction « sendProductFashionSeeker » que j'avais créée dans mon service, et ensuite j'ai récupéré l'image au format Grand Zoom. J'ai pu trouver des exemples de ce code qui était déjà présent dans le projet. Ensuite, j'ai pu envoyer l'URL à la fonction du service et appeler une deuxième fonction « getProductFromProductIds » qui était déjà créée pour récupérer les produits en fonction des identifiants des produits. J'ai ensuite créé un tableau pour stocker les identifiants de produits obtenus, puis enfin j'ai fait appel à la fonction pour récupérer les produits en passant le tableau d'identifiants créé et je récupère les produits et je les range dans une constante créée avant la fonction. La fonction terminée je l'appelle dans un watcher qui va se déclencher quand je veux ouvrir la modale.

Pour l'affichage dans le template, j'ai créé deux parties, une qui affiche un message s'il n'y a pas de produits trouvés, j'ai pour cela créé une constante « hasSimilarProducts » qui se met à false si la longueur du tableau de productIds créé est égale à 0. La deuxième partie, elle, s'affiche quand il y a des produits et renvoie la liste de produits similaires.

Quand tout cela a été terminé, j'ai alors effectué une démonstration devant plusieurs membres de l'équipe e-commerce pour avoir un retour sur ce que j'ai fait. Plusieurs points à améliorer sont ressortis de cette démonstration :

- Premièrement, il faudrait mettre en place un skeleton (Voir Annexe 2 ci-dessous), il remplace ce que l'on appelle un loader pour attendre le chargement des produits sur la page.

- Deuxièmement, il faudrait afficher l'image que l'on a envoyée et montrer les objets détectés, ensuite on peut cliquer sur un objet pour trier les produits similaires en fonction de l'objet détecté auquel il appartient.

À la suite de cette réunion, il m'a fallu mettre en place ces deux fonctionnalités.

Pour le skeleton, c'était plutôt simple à mettre en place car le composant affichant les zones grises était déjà créé. Ce qui a été le plus long sur cette partie a été le SCSS que j'ai mis en place pour que les zones prennent la même taille que les produits de la modale. De plus, pour que le temps de chargement soit visible, j'ai créé une nouvelle constante « `isLoading` » que j'ai rajoutée dans ma fonction « `fetchSimilarProduct` », `isLoading` est mis à `true` dès le début de la fonction et on la met à `false` s'il ne trouve pas de produits ou si la recherche de produits est terminée.

Pour la deuxième fonctionnalité c'était plus technique car j'ai dû rechercher un moyen de faire apparaître les zones des objets détectés sur une image, j'ai d'abord pensé à `<canvas>` avec du Javascript mais je ne savais pas si cela serait adapté pour une image, mais en continuant de chercher j'ai trouvé d'autres propriétés HTML : `<map>` et `<area>` qui me permettaient directement de faire des zones cliquables sur une image grâce à des coordonnées. En sachant, que dans la réponse que je recevais de l'API Vision quand j'envoyais une image, il y avait les coordonnées de l'objet sur l'image, j'ai pensé que c'était une bonne idée d'utiliser ces deux propriétés.

Format de la réponse de l'API :

```
{
  "objectsDetected": [{
    "label": "dress",
    "score": 0.5,
    "position": {
      "x": 5,
      "y": 2,
      "width": 10,
      "height": 20
    },
    "similarProducts": [{
      "productId": "11210",
      "score": 0.5
    }]
  }]
}
```

J'ai donc modifié le composant affichant les images pour rajouter l'affichage des zones. J'ai d'abord commencé par rajouter des props pour faire en sorte de limiter l'utilisation de la détection de zones à seulement les images situées dans le

composant du « Fashion Seeker », de plus j'ai passé au composant une autre props qui me permet de transférer les objets détectés au composant pour récupérer les coordonnées.

Pour ce qui est de l'utilisation de `<map>` et `<area>` j'ai besoin de plusieurs choses. J'ai ici un exemple simple de l'utilisation de `<map>` et `<area>` :

```
<map name="primary">
  <area shape="circle" coords="75,75,75" href="left.html" />
  <area shape="circle" coords="275,75,75" href="right.html" />
</map>

```

On a donc besoin pour le `<map>` d'un « name » qui sera le même qui sera utilisé dans le usemap attaché à l'image et pour les `<area>` nous aurons besoin de la forme, celle que j'ai utilisée dans le projet étant

« rect » pour un rectangle et des coordonnées des quatre sommets du rectangle.

Mais avant tout cela j'ai dû faire certaines conversions et changements car j'ai rencontré plusieurs problèmes, tout d'abord area n'est pas directement modifiable avec du SCSS j'ai donc dû créer une `<div>` avec un style sur chaque côté et une position absolue pour délimiter les zones clairement. Mon deuxième problème était beaucoup plus complexe, quand j'envoyais une image au « Fashion Seeker » peu importe le format de l'image que j'envoyais, la réponse retournait des coordonnées pour des images de tailles beaucoup plus grandes par rapport à l'image affichée, j'ai d'abord essayé de régler le problème en faisant des conversions de taille en prenant la `clientWidth` et la `clientHeight` qui correspond à la taille de l'image affichée sur le site web et de les diviser par la `naturalWidth` et `naturalHeight` de l'image c'est-à-dire la taille intrinsèque mais c'est celle-ci qui me posait un problème car la taille intrinsèque n'était pas la taille de l'image pour laquelle je récupérais les coordonnées. J'ai donc dû demander de l'aide à Julien Dubus qui est la personne qui a mis en place l'API Vision pour pouvoir faire en sorte que certaines conversions soient faites directement dans le backend et que le « Fashion Seeker » prennent aussi en compte les images de grandes tailles pour lesquelles les objets détectés n'avaient pas les bonnes positions, suite à cela j'ai toujours un petit problème de positionnement des cadres j'ai donc pour régler le problème créé une nouvelle props pour enlever les `sourceSets` qui faisaient buguer le code ici car les images n'avaient plus la même taille intrinsèque.

Après cela j'ai pu calculer les coordonnées facilement pour les quatre points (cela donne `x`, `y`, `x+width` et `y+height` car la réponse de l'API ne rendait pas les quatre sommets). J'ai ensuite effectué toutes mes conversions de tailles (Voir Annexe 3 ci-dessous) sur les zones pour les ajuster, mais j'ai aussi réalisé une autre conversion

car j'ai recadré les images pour qu'elles fassent toute la même taille avec un 'object-fit : cover'.

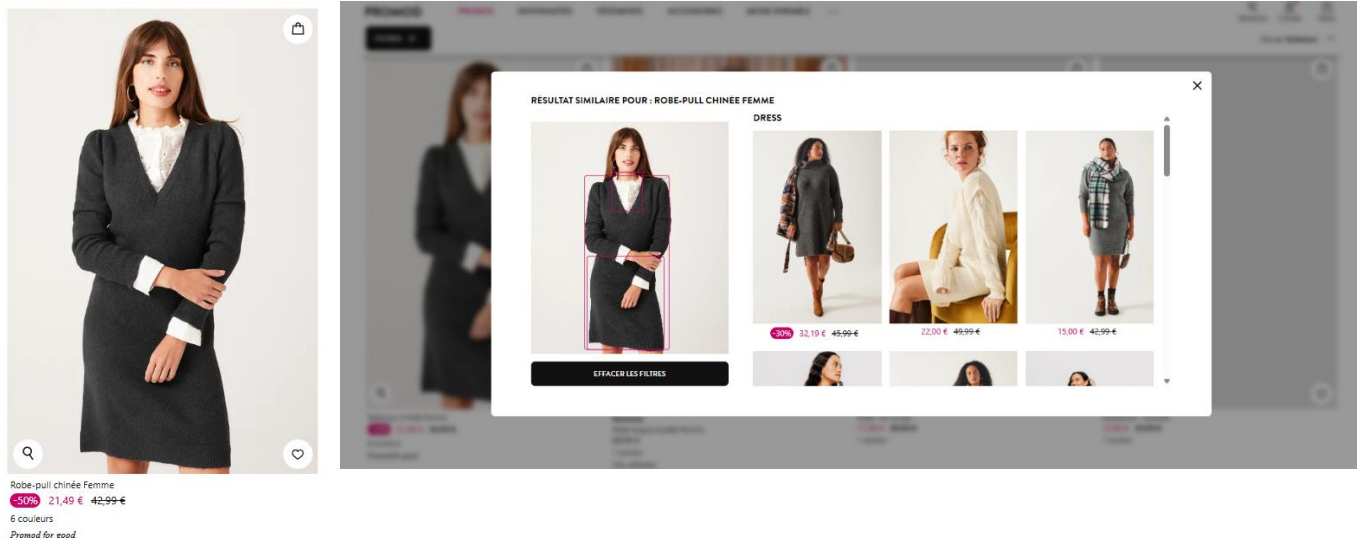
Pour finir avec le composant image, j'ai ajouté une classe qui permet de faire en sorte que quand je passe ma souris sur l'area, le SCSS de la <div> qui affiche le cadre change car si je le change directement avec un hover je ne pouvais plus accéder au lien sur <area> et j'ai rajouté une fonction qui émet un événement quand je clique sur la zone, j'appelle cette fonction avec un addEventListener dans un onMounted qui vient du Vuejs et qui est appelé quand le composant a fini d'être monté. J'ai mis aussi dans ce onMounted un addEventListener qui va traquer le changement de taille de la fenêtre pour recalculer les proportions des zones.

Et j'ai malheureusement aussi un autre problème que je ne pense pas pouvoir régler pour l'instant car au niveau de l'affichage des zones j'ai parfois des zones qui se superposent et qui empêchent l'accès aux autres zones, j'ai malgré tout essayé de réfléchir à la façon de régler le problème et je pense que l'on pourrait calculer l'aire des zones et mettre en avant les zones avec la plus petite aire, je ne pense pas que cela réglerait tous les problèmes mais c'est une piste à explorer pour régler cela.

Enfin, cela nous ramène vers le composant de la modale où grâce à l'événement emit que j'ai créé précédemment j'appelle une nouvelle fonction qui va faire en sorte que quand je clique sur une zone cela n'affiche que les produits de cette zone, mais avant ça il y a un problème à régler, à ce moment-là tous les produits sont toujours dans le même tableau peu importe leur genre car les produits sont en réalité classés par éléments détectés comme par exemple les robes avec le label « dress », les t-shirts avec le label « Top ». Il faut alors les classer, et pour cela je crée des groupes de tableau qui ont chacun un label et la liste de produits correspondant au label et je remplace cette façon de trier les produits directement dans la fonction « fetchSimilarProduct » (voir Annexe 4 ci-dessous). Ce qui me permet maintenant de filtrer les produits en fonction de la zone sur laquelle je clique.

Avec cela la partie du « Fashion Seeker » où on utilise les images déjà présentes sur le site est presque fini. J'ai juste rajouté en dessous de l'image un bouton qui permet de réinitialiser les filtres que l'on a appliqués.

Voici le rendu de cette première version du « Fashion Seeker » :



2. Le Fashion Seeker avec l'upload d'un fichier

Cette version du « Fashion Seeker » peut générer un risque au niveau de l'utilisation car cela peut être utilisé pour de la comparaison de produits par des entreprises concurrentes ou il se pourrait que des images inappropriées soient uploadées. Donc cette version ne sera probablement pas intégrée au site web mais j'ai tout de même codé cette version qui n'est pas si différente de l'autre mais plusieurs changements vont être effectués.

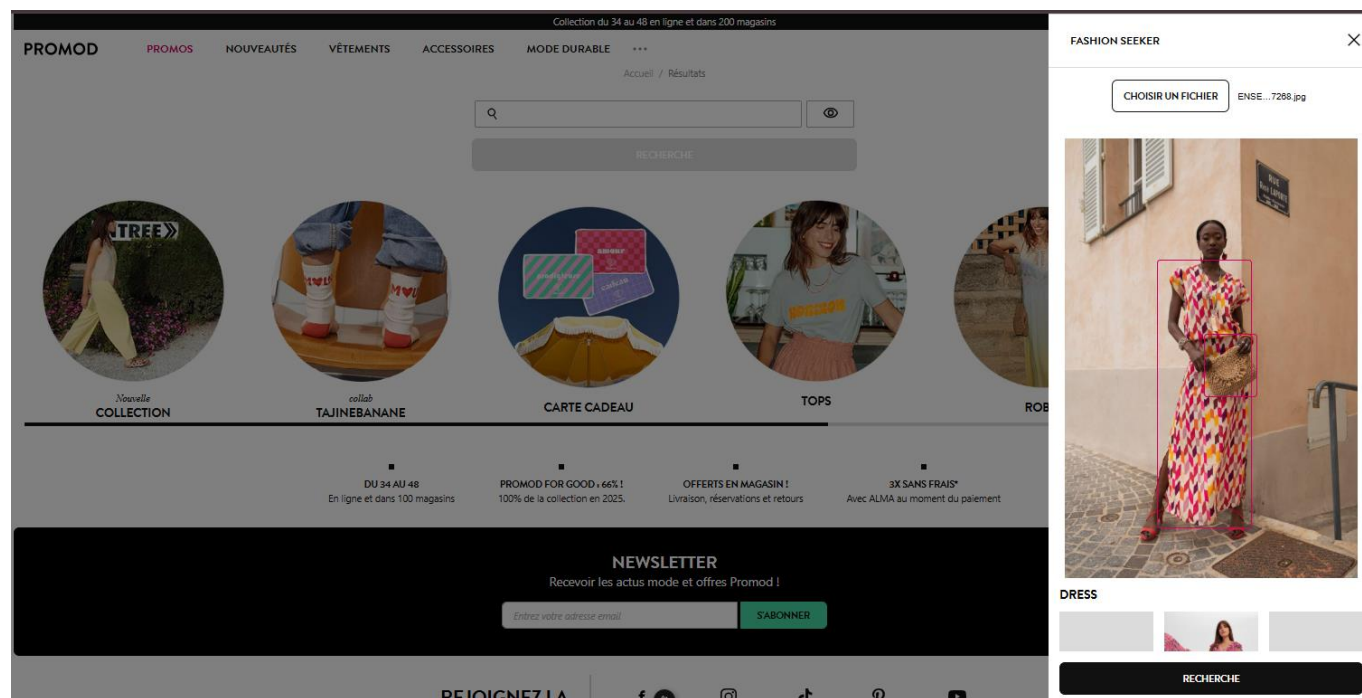
Premièrement le plus gros changement effectué est que l'on ne va plus envoyer l'URL de l'image dans l'URL mais on va passer un fichier par le corps de la requête. Donc dans ce composant pour la version de la recherche nous n'avons plus besoin de props pour récupérer le produit mais nous aurons besoin de récupérer l'image avec un formulaire, et on va appeler la fonction `fetchSimilarProduct` quand va soumettre le formulaire et on va cependant lui passer l'image en paramètre.

Pour que la requête aboutisse, il y aura aussi des changements au niveau du service, où l'on va utiliser le « filename » au lieu de l'URL de l'image. On va alors le passer dans le body sous la forme d'un `formData`.

Je n'ai pas réalisé beaucoup de changements au niveau de la fonction pour avoir les produits similaires mais j'ai dû adapter l'affichage de l'image que l'on a envoyée et qui est aussi l'image où l'on affiche les zones car je ne peux pas les afficher directement. J'ai alors trouvé une solution en utilisant `URL.createObjectURL` et en lui mettant en paramètre l'image que j'ai envoyée, cela crée un URL d'objet de type blob, mais si je la laisse comme cela l'image ne va pas s'afficher car cette technique bloque avec des paramètres. J'ai alors créé un if dans une fonction qui en gros ne met pas de paramètre quand l'URL commence par « blob : ». Avec tout cela mon

composant de recherche visuelle avec une image importée marche. J'ai aussi changé une autre chose par rapport à l'autre composant qui lui, était affiché dans une modale. J'ai utilisé un composant « Sliding Layer » présent dans le code déjà fourni qui m'a permis de mieux intégrer l'encadrement pour l'upload du fichier car je n'avais pas d'idée de placement pour chacun des éléments du composant dans la modale utilisée avant.

Voici le rendu de la deuxième version du « Fashion Seeker » :



3. Les tests

Une dernière chose que j'ai dû réaliser lors de ce stage et dont je n'ai jamais entendu parler avant est le testing, c'est-à-dire que l'on teste différents éléments ou fonctionnalités du composant pour savoir s'ils vont fonctionner dans certaines circonstances. Dans le projet, j'ai dû utiliser l'environnement de test Vitest et Test-Utils. Pour illustrer mon propos, j'ai réalisé un test sur la récupération des produits similaires. Pour ce faire, j'ai dû créer des mocks pour simuler des données. Ici j'ai créé un mock de produit et un mock d'une réponse de l'API Vision. Dans ce test je vérifie d'abord si mes deux fonctions « sendProductFashionSeeker » et « getProductFromProductIds » ont été appelées, puis je regarde si « hasSimilarProduct » est à true pour savoir s'il y a des produits similaires. Je regarde aussi si l'état de chargement est mis à false et si la longueur du tableau d'objets détectés est au moins à 1. Tout cela me permet de tester ma fonction pour savoir si elle marche en toute circonstance. Le testing est encore quelque chose d'un peu

flou pour moi et j'aimerais en apprendre plus sur ce domaine car cela a l'air d'être un plus pour la sûreté du code.

Test sur la récupération des produits :

```
test("doit récupérer les produits similaires et mettre à jour un état correctement", async () => {
  const mockResponse = {
    objectsDetected: [
      {
        label: "dress",
        score: 0.5,
        position: { x: 5, y: 2, width: 10, height: 20 },
        similarProducts: [{ productId: "456", score: 0.7 }],
      },
    ],
  };

  sendProductFashionSeekerMock.mockResolvedValue(mockResponse);
  getProductFromProductIdsMock.mockResolvedValue([{ productId: "456" }]);

  await wrapper.vm.fetchSimilarProducts();

  expect(sendProductFashionSeekerMock).toHaveBeenCalled();
  expect(getProductFromProductIdsMock).toHaveBeenCalled();
  expect(wrapper.vm.hasSimilarProducts).toBe(true);
  expect(wrapper.vm.isLoading).toBe(false);
  expect(wrapper.vm.groupedSimilarProducts[0].label).toBe("dress");
  expect(wrapper.vm.objects.objectsDetected).toHaveLength(1);
});
```

III. CONCLUSION

Ce stage m'a permis d'atteindre les objectifs fixés au départ, j'ai réussi à participer au développement du site e-commerce par le biais du projet « Fashion Seeker ». Le projet était très intéressant pour mon parcours car j'ai pu découvrir d'autres langages comme le TypeScript, mais aussi Nuxt 3, qui m'a permis de développer des compétences avec ce framework mais aussi indirectement avec le framework Vuejs car Nuxt se base sur celui-ci. Je pense que même s'il y a des éléments similaires entre ce que j'ai fait lors de mes études et ce que j'ai fait lors de mon stage, il y a quand même une différence drastique entre les deux, car ce que j'ai fait lors de mon stage semble beaucoup plus concret que ce que j'ai fait en classe et cela servira plus probablement à quelque chose de réel (ici, cette fonctionnalité pourrait être intégrée sur le site.). Ce que j'ai appris lors de mes études a quand même eu une utilité dans le sens que cela m'a permis d'assimiler plus vite le code pour comprendre et pouvoir réaliser le projet.

Le stage m'a aussi permis de remplir l'objectif de participer à certains rituels d'entreprise et de comprendre comment marche le monde du travail. En effet, je pense que grâce aux rituels agiles SCRUM, il y a une vraie méthodologie à suivre et il n'y a pas de dérive comme pour certains projets lors des études où on définissait un tableau de tâches à faire et finalement les tâches étaient partiellement réalisées voire pas du tout, ou des tâches réalisées mais pas dans l'ordre que l'on a fixé à la base. Je pense qu'au niveau des études les méthodes agiles devraient être plus définies car elles restaient jusqu'alors très floues.

Pour ce qui est des difficultés rencontrées durant le stage, je pense que l'un des plus gros défis a été d'abord de comprendre le code et de s'adapter, car je n'avais peu voire pas de connaissance sur le framework, cela m'a permis de m'améliorer avec d'autres langages.

Pour ce qui en revanche a été facile, c'était l'intégration dans l'équipe tant sur le plan professionnel que sur le plan personnel, en participant par exemple aux activités extra-professionnelles comme un Laser Game.

De plus, ce stage m'a permis de gagner en autonomie, car j'ai réalisé en grande partie seule le travail lié au front-end, tout en sollicitant ponctuellement de l'aide sur les aspects avec lesquels j'étais moins à l'aise ou pour résoudre certains bugs complexes.

Cette expérience a été instructive et gratifiante, et je pense qu'elle représente une très bonne expérience pour mon parcours professionnel.

ANNEXES

Annexe 1-Comparaison Data Fetching Nuxt 3 et Axios

Nuxt 3

```
<script setup lang="ts">
const { data } = await useFetch('/api/data')

async function handleFormSubmit() {
  const res = await $fetch('/api/submit', {
    method: 'POST',
    body: {
      // My form data
    }
  })
}
```

Axios

```
axios.post('/user', {
  firstName: 'Fred',
  lastName: 'Pierrafeu'
})
.then(function (response) {
  console.log(response);
})
.catch(function (error) {
  console.log(error);
});
```

Ici deux exemples de requêtes POST, une sous Nuxt 3 et une avec axios.

Annexe 2- Exemple de skeleton



Voici un exemple de skeleton qui est en vogue en ce moment pour remplacer les loaders classiques.

Annexe 3- Code de conversion pour les tailles d'images

```
const getCoords = (obj: any): string => {
  const { x, y, width, height } = obj.position;

  const x1 = x * scale.value - offsetX.value;
  const y1 = y * scale.value - offsetY.value;
  const x2 = (x + width) * scale.value - offsetX.value;
  const y2 = (y + height) * scale.value - offsetY.value;

  return `${x1},${y1},${x2},${y2}`;
};

const calculateScale = () => {
  ImageFS.value = document.getElementById("scale");
  if (
    ImageFS.value.naturalWidth &&
    ImageFS.value.naturalHeight &&
    ImageFS.value.naturalWidth !== 1
  ) {
    const containerWidth = ImageFS.value.clientWidth;
    const containerHeight = ImageFS.value.clientHeight;
    const imageRatio = ImageFS.value.naturalWidth / ImageFS.value.naturalHeight;
    const containerRatio = containerWidth / containerHeight;

    if (imageRatio > containerRatio) {
      scale.value = containerHeight / ImageFS.value.naturalHeight;
      offsetX.value =
        (ImageFS.value.naturalWidth * scale.value - containerWidth) / 2;
      offsetY.value = 0;
    } else {
      scale.value = containerWidth / ImageFS.value.naturalWidth;
      offsetY.value =
        (ImageFS.value.naturalHeight * scale.value - containerHeight) / 2;
      offsetX.value = 0;
    }
  }
};
```

La première fonction calcule les coordonnées et la deuxième calcule les valeurs de scale pour la conversion de tailles, et offset pour le recadrage.

Annexe 4- Code de la fonction de récupération des produits similaires

```
const fetchSimilarProducts = async () => {
  isLoading.value = true;
  if (!props.product) {
    hasSimilarProducts.value = false;
    isLoading.value = false;
    return;
  }
  const { sendProductFashionSeeker } = usePmdFashionSeekerServices();
  const runtimeConfig = useRuntimeConfig().public;
  const filterImg = props.product.pictures?.find(
    (image) => image.type === PictureTypeEnum.GRAND_ZOOM
  );
  if (filterImg) {
    const urlImage = runtimeConfig.assetBaseUrl + filterImg.path;
    try {
      const response = await sendProductFashionSeeker(undefined, urlImage);
      const paramsStore = useParamsStore();
      const { getProductFromProductIds } = usePmdProductServices();
      const grouped: { label: string; products: IProduct[] }[] = [];
      objects.value = response;
      starterImg.value = urlImage;
      for (const detectedObject of response.objectsDetected) {
        const productIds = detectedObject.similarProducts.map(
          (product: any) => product.productId
        );
        let products =
          (await getProductFromProductIds(productIds, paramsStore.storeId)) ||
          [];
        products = products.filter(
          (product: IProduct) => product?.productId !== props.product?.productId
        );
        if (products.length > 0) {
          grouped.push({
            label: detectedObject.label,
            products,
          });
        }
      }
      groupedSimilarProducts.value = grouped;
      allGroups.value = grouped;
      hasSimilarProducts.value = grouped.length > 0;
    } catch {
      hasSimilarProducts.value = false;
    }
  } else {
    hasSimilarProducts.value = false;
  }
  isLoading.value = false;
};
```

Ici on retrouve ma fonction qui sert à récupérer les produits similaires avec l'URL d'une image, on voit qu'on applique directement l'état de chargement dès le début, puis en gros on appelle les méthodes des services et on récupère l'URL de l'image, puis on envoie l'url et on stocke la réponse productIds et on envoie ce tableau d'identifiant à la deuxième méthode pour avoir les produits que l'on met dans grouped, ensuite on met grouped dans deux ref extérieures allGroups et groupedSimilarProducts puis on enlève l'état de chargement.

Annexe 5 - Code pour le service usePmdFashionSeekerServices

```
import type { IProduct } from "~/types/api/product/product.interface";

export const usePmdFashionSeekerServices = (): {
  sendProductFashionSeeker: (
    filename?: File,
    urlImage?: string
  ) => Promise<IProduct[]>;
} => {
  const sendProductFashionSeeker = async (
    filename?: File,
    urlImage?: string
  ): Promise<IProduct[]> => {
    let uri = `/vision`;
    if (urlImage) {
      uri += `?url-image=${urlImage}`;
    }

    const formData = new FormData();
    if (filename) {
      formData.append("filename", filename);
    }

    return usePmdAPI(encodeURI(uri), {
      method: "POST",
      body: formData,
    })
      .then((response: any) => response)
      // .catch((_error: FetchError) => {
      //   // const { $logger } = useNuxtApp();
      //   // $logger.error({
      //   //   message:
      //   //     'Erreur lors de l\'appel à la méthode "sendProductFashionSeeker" du service vision',
      //   //   error: error.value,
      //   // });
      //   // return [];
      // });
  };

  return {
    sendProductFashionSeeker
  };
};
```

Ici c'est le service, dans le code on appelle la méthode « sendProductFashionSeeker ». Comme on le voit dans ce code on peut soit choisir « filename », soit « urlImage » car ils sont tous les deux suivis d'un point d'interrogation voulant signaler qu'ils sont optionnels. Et en fonction de ce qu'on utilise on le passe soit par l'URL, soit par le body dans un formData.